

CS 2006 – Operating Systems
Spring 2026



CHRONO RIFT

PROJECT DOCUMENTATION

Gri Raj
23I-0601

1. Project Overview

The game Chrono Rift operates as a tactical turn-based game which uses multiple processing units and multiple simultaneous execution threads to run its software on Linux. The game demonstrates essential operating system functions through its implementation of process isolation and shared memory inter-process communication and multi-threading which requires synchronization and its stamina-based scheduling system and asynchronous signal delivery mechanism and deadlock detection system and its inventory management system which uses contiguous memory allocation.

The game runs entirely inside a Docker container on WSL 2 through its use of Linux system calls which include `shm_open` and `mmap` and `pthread`s and `semaphores` and `POSIX` signals. All five executables are compiled via a single Makefile and launched through the arbiter process.

2. Process Architecture & IPC

The game uses three distinct processes which interact with each other through a shared memory segment that follows POSIX standards. Pipes are explicitly prohibited and not used anywhere in the codebase. The arbiter process operates the SFML graphical renderer through a separate rendering thread.

Process	Role	Key OS Concepts
arbiter	Central authority: owns shared memory, runs scheduler, manages signals, handles deadlock	fork/exec, shm_open, mmap, SIGUSR1, SIGALRM, SIGTERM
human_process	Captures keyboard input; one pthread per player; dispatches turns	pthread, semaphores, active player threads, SIGUSR1
strategic_process	AI decision-making; one pthread per NPC; concurrent NPC logic	pthread, semaphores, suspended by SIGSTOP / SIGCONT
render (arbiter thread)	SFML graphical window; dedicated render thread inside arbiter; reads shared memory	SFML Graphics, pthread, sem_wait, 10 fps render loop

2.1 Shared Memory Layout

The game state information which includes entity statistics and inventory details and artifact table and action mailboxes and action log data and all synchronization elements (`sem_t` with `pshared=1`) exists within a single `SharedState` struct which the system maps through `shm_open` and `mmap` using `MAP_SHARED`. The system does not implement any usage of pipes FIFOs or sockets throughout its entire operation.

3. Synchronization & Concurrency Control

Semaphore	Purpose	Init Value
state_mutex	Protect full SharedState during read/write	1 (mutex)
action_sem	Arbiter blocks here; entity posts when action ready	0
turn_sem	Entity blocks here; arbiter posts when entity's turn starts	0
artifact_mutex	Protect artifact resource table exclusively	1 (mutex)
log_mutex	Protect action log ring buffer	1 (mutex)

All semaphores exist as unnamed entities which use `sem_init` with `pshared=1` to create semaphores that operate within the shared memory segment thus meeting the requirement of 'memory-based primitives' system design. The system protects against race conditions because all operations that access shared state require the execution of `sem_wait` and `sem_post` functions on the `state_mutex`. The UI rendering thread takes a full snapshot under the lock and immediately releases the lock which reduces contention.

4. Multithreading & Execution Model

Process	Threads Created	Description
human_process	1 dispatcher + N player threads (N = 1–4)	Turn dispatcher wakes the correct player thread via per-thread semaphore
strategic_process	1 dispatcher + M NPC threads (M = 2–9)	Each NPC runs its AI in its own thread; concurrent decisions before serialisation
arbiter	1 deadlock monitor thread	Background thread continuously checks artifact wait-for graph for cycles
render	1 rendering thread	Reads shared memory snapshot at 10 fps; draws SFML graphical window

All NPC threads run concurrently because they operate independently without needing serialization into one single thread. The NPC uses its dedicated thread to determine which action to execute. The Arbiter enforces serial action execution through the stamina scheduler which controls execution at that level instead of stopping multiple threads from running at the same time.

5. Stamina-Based Scheduling & Temporal Logic

The scheduler operates according to a time-based system which enables perpetual stamina accumulation. The system provides each living entity with speed points of stamina for every second of existence. An entity is eligible to act when stamina reaches or exceeds its maximum capacity. The first entity which achieves complete stamina will perform an action while its index number will determine which entity performs first in case of a tie. After performing an action, the actor loses all his stamina except for Skip which maintains half of his energy. The other entities maintain their existing stamina levels, which allows quicker entities to receive additional turns.

Entity Type	Max Stamina	Speed	Typical Turn Cycle
Player (N players)	100	100 / N	N players share equal speed
Enemy	150	Random 10–30	Faster enemies act more frequently

Serial execution: while stamina accumulates in parallel for all entities, only one entity acts at any given moment. The arbiter posts to `turn_sem` to wake the entity, waits on `action_sem` for the response, then applies the action atomically.

6. Signals & Asynchronous Interrupts

Signal	Sender	Receiver	Effect
SIGUSR1	Arbiter	Human / Strategic process	Stun: pause for exactly 3 seconds; stamina zeroed
SIGTERM	Human process	Arbiter	Quit condition: Arbiter sets RESULT_QUIT, cleans up
SIGSTOP	Arbiter	Strategic process	Suspend all NPC threads (Ultimate ability, 10 s)
SIGCONT	Arbiter (SIGALRM handler)	Strategic process	Resume after 10 s Ultimate window
SIGALRM	Kernel (alarm(10))	Arbiter	Fires after 10 s; handler resumes Strategic, posts turn_sem

The system manages to deliver the SIGUSR1 signal to the targeted process during non-blocking operation. The signal handler uses sleep(3) to pause execution time and it also removes the stunned status because the target process does not check either pipes or flags during its primary operation. The attacking process does not wait for confirmation because it continues to execute without interruption. The ultimate suspension process requires only SIGSTOP and SIGCONT signals without using any pipes or flags.

7. Deadlock Handling & Resource Management

There are three global artifacts that function as exclusive resources which include Solar Core and Lunar Blade and Eclipse Relic. The Global Resource Table in shared memory maintains records of each artifact's status which shows whether it is FREE or HELD and identifies the entity that holds it. The table can be accessed through the artifact_mutex system which protects its contents.

7.1 Deadlock Detection

The Arbiter background deadlock monitor thread operates every second. The thread creates a wait-for graph by examining the resource table through its inspection process: entity A who possesses Solar Core and entity B who possesses Lunar Blade ($A \rightarrow B$) create a circular wait situation (both need to possess each other's artifact for their Ultimate to activate). The monitor requires the entity with lower index to cease possession of its current artifact because this action will eliminate the deadlock situation.

7.2 Eclipse Relic

The Eclipse Relic is introduced dynamically at runtime when a character picks it up from the environment. It is added to the Global Resource Table at that moment and follows the same exclusivity and locking rules as the other artifacts.

8. Memory Management – Inventory Allocator

The player's primary inventory is a linear array of 20 slots. The allocator implements contiguous-fit allocation (first-fit), analogous to OS memory allocation:

Operation	Algorithm	Complexity
Pick up weapon (size N)	Scan for first run of N free slots; place weapon	$O(\text{SLOTS})$
Out of space	Swap out minimum weapons to LTS until N contiguous slots free	$O(\text{WEAPONS} \times \text{SLOTS})$
Swap In from LTS	Remove from LTS, then apply pick-up protocol	$O(\text{LTS} + \text{SLOTS})$
Remove weapon	Clear all slots occupied by weapon (first-slot traversal)	$O(\text{SLOTS})$

The Solar Core and Lunar Blade combination occupies all 20 available weapon slots which results in complete weapon storage capacity being reached because they occupy two weapon slots. The slot sizes of the system establish this requirement as a fundamental rule which needs to be followed. The allocator maintains system functionality by replacing weapons through minimum necessary swaps to establish a continuous vacant space in the process of handling the two-slot Splinter Stick.

8.1 Weapon Table

Weapon	Slots	Damage	Artifact?
Solar Core	10	95	Yes (exclusive)
Lunar Blade	10	90	Yes (exclusive)
Eclipse Relic	5	60	Yes (dynamic)
Iron Halberd	7	55	No
Thunderstaff	6	50	No
Frostbow	6	48	No
Obsidian Axe	5	45	No
Venom Dagger	4	30	No
Splinter Stick	2	12	No

9. Gameplay Logic & Mechanics

9.1 Entity Stats (Roll-Number Personalised)

Stat	Player	Enemy
HP	Roll No + random(100, 1000)	Last 2 digits of Roll No + random(50, 200)
Damage	Last digit of Roll No + 10	Second-last digit of Roll No + 10
Speed	100 / number of players	random(10, 30)

Stat		Player	Enemy
Max Stamina	100		150

9.2 Player Actions

- Strike — direct HP damage to target
- Exhaust — reduce enemy stamina
- Use Weapon — apply weapon damage
- Swap In — retrieve weapon from LTS (costs turn)
- Heal — restore 10 % HP
- Skip — 50 % stamina refund
- Ultimate Ability — available when both Solar Core and Lunar Blade are in primary inventory

9.3 Enemy Actions

- Strike — target lowest-HP player (80 % probability)
- Skip — 20 % probability

9.4 End Conditions

- Win — kill 10 enemies total
- Lose — all player characters die
- Quit — player selects Quit; Human Process sends SIGTERM to Arbiter

9.5 Randomisation Seed

The player's roll number is used as the LCG seed ($\text{seed} = \text{seed} \times 1664525 + 1013904223$) for all random values: number of enemies (2–9), HP, speed, stun chance, and NPC AI decisions.

10. Rendering & System Integration

The arbiter process uses a separate SFML thread to perform rendering functions. The program establishes an `sf::RenderWindow` which limits its frame output to 10 frames per second. The program first takes control of `state_mutex` to create a local copy of `SharedState` and then unlocks the mutex to begin drawing operations. This design prevents the user interface from stopping the operation of the scheduler. The system displays a splash screen which remains active until the user presses a key or clicks their mouse.

The SFML window displays:

- HP bars per entity (colour-coded: green for players, red for enemies)
- Stamina bars per entity
- Artifact resource table with hold status
- Action log (last 8 entries)
- Phase and kill-count status
- Splash screen on startup with title and animated prompt

11. Turnaround Time Analysis

In the stamina scheduler, each entity’s ‘turnaround time’ is the number of seconds between consecutive actions.

Formula: arrival_time = max_stamina / speed (seconds to fill from empty). After acting (stamina reset to 0): next_turn = max_stamina / speed seconds later. After Skip (stamina reset to 50 %): next_turn = (max_stamina × 0.5) / speed seconds later.

Example (4 players, enemy speed=20)	Turn Cycle (s)	Skip Cycle (s)
Player (speed = 25)	100/25 = 4.0 s	50/25 = 2.0 s
Enemy (speed = 20)	150/20 = 7.5 s	75/20 = 3.75 s
Enemy (speed = 30)	150/30 = 5.0 s	75/30 = 2.5 s

With 1-second ticks, the actual cycle is [max_stamina / speed] ticks. Faster entities (high speed relative to max_stamina) act more frequently, creating the asymmetric temporal combat described in the project spec.

12. Build Instructions & Deployment

12.1 Requirements

- WSL 2 (Ubuntu 22.04 or 24.04) or any Linux host with X11 / display server
- Docker Desktop with WSL 2 backend enabled
- gcc/g++, make, libsFML-dev (pre-installed in the Docker image)

12.2 Native Build (Ubuntu 24 / WSL 2)

```
make all
./chrono_rift
```

12.3 Docker Build & Run

```
docker build -t chrono_rift .
docker run --rm -e DISPLAY=$DISPLAY -v /tmp/.X11-unix:/tmp/.X11-unix --network host chrono_rift ./arbiter_bin
```

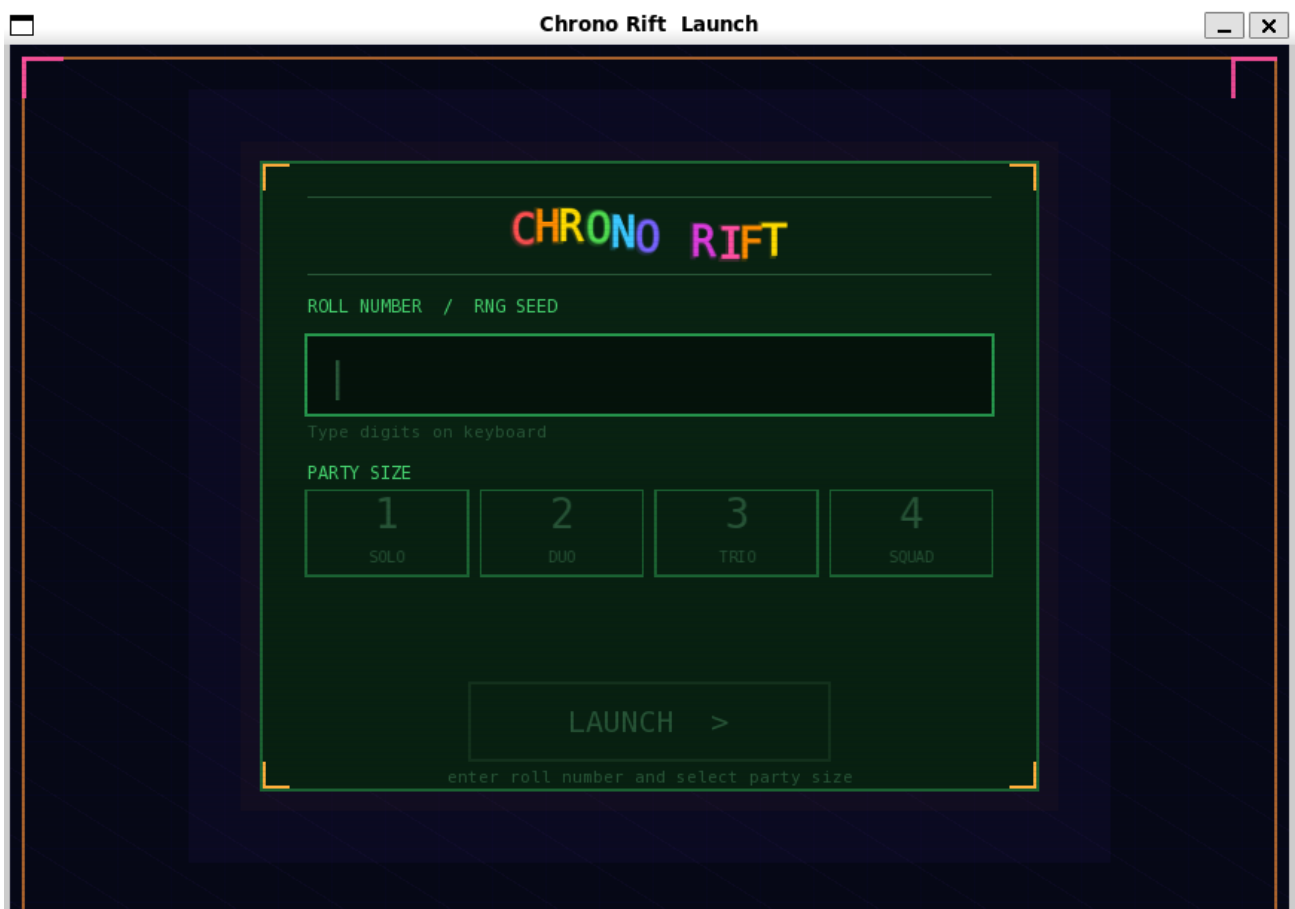
The -e DISPLAY and X11 socket volume mount are required because SFML opens a graphical window via the host X server. The WSL 2 environment requires users to start an X server which can be either VcXsrv or WSLg before they can begin using the container.

13. Project File Structure

File	Description
Makefile	Builds all 5 executables; clean target
Dockerfile	Ubuntu 22.04 base; installs g++, make, libsFML-dev
requirements.txt	Documents system library dependencies
arbiter/shared_state.h	All shared types: Entity, SharedState, WeaponDef, semaphores
arbiter/inventory.h	Inventory allocator API
arbiter/main_launcher.cpp	Launcher: prompts roll no + party size, forks arbiter
arbiter/arbiter.cpp	Arbiter: scheduler, signal handling, deadlock monitor, SFML render thread
arbiter/sprite_draw.inl	Inline SFML rendering helpers: draw_bar_sfml, draw_panel, entity rows
hip/hip.cpp	Human Interfacing Process: player threads, keyboard input
asp/asp.cpp	Automated Strategic Process: NPC threads, AI logic
arbiter/inventory.cpp	Contiguous allocator: first-fit, swap-out, swap-in

All source files follow professional coding standards: consistent indentation (4 spaces), descriptive variable and function names, section comments, and clear separation of concerns across files.

14. Screenshots



Player-2's Turn

Choose Action

P = menu

Player-2

HP

1217/1217

ST

100/100

DMG: 11 SPD: 50

INVENTORY

(empty)

1 Strike

Direct damage to enemy

2 Exhaust

Drain enemy stamina

3 Use Weapon

Attack with inventory weapon

4 Swap In

Retrieve from Long-Term Storage

5 Heal

Restore 10% of max HP

6 Skip

50% stamina refund

0 Quit Game

Send SIGTERM to arbiter

ENEMIES

Enemy-1 [DEAD]

Enemy-2

155/166

ST 150

Enemy-3

95/95

ST 150

Enemy-4

138/138

ST 150

Enemy-5

181/181

ST 150

Enemy-6

56/56

ST 150

Enemy-7

95/95

ST 150

Enemy-8

200/200

ST 150

Enemy-9

140/140

ST 150

CS 2006 – Spring 2026

Page 10

CHRONO RIFT

Kills : 0 / 10 RUNNING

PLAYERS

Name	Status	HP Bar	HP	Stamina Bar	ST
Player-1			782/802		66/100
Player-2			1217/1217		33/100
Player-3			1552/1552		100/100

ENEMIES

Enemy-1	[STUN]		79/200		150/150
Enemy-2			95/95		84/150
Enemy-3			144/144		150/150
Enemy-4			54/54		150/150
Enemy-5			112/112		150/150
Enemy-6			134/134		150/150
Enemy-7			185/185		150/150
Enemy-8			55/55		150/150
Enemy-9			105/105		150/150

ARTIFACTS

Solar Core : FREE

Lunar Blade : FREE

ACTION LOG

```

>>> Enemy-1's turn (stamina full)
Enemy-1 skips
>>> Player-1's turn (stamina full)
Player-1 strikes Enemy-1 for 11 dmg
>>> Player-2's turn (stamina full)
Player-2 strikes Enemy-1 for 11 dmg
>>> Player-3's turn (stamina full)
Player-3 strikes Enemy-1 for 11 dmg
>>> Enemy-2's turn (stamina full)
Enemy-2 strikes Player-1 for 10 dmg
>>> Player-1's turn (stamina full)
Player-1 strikes Enemy-1 for 11 dmg
>>> Player-2's turn (stamina full)
Player-2 strikes Enemy-1 for 11 dmg
Enemy-1 is STUNNED for 3 seconds!
>>> Player-3's turn (stamina full)

```